

ΝΕΥΡΩΝΙΚΑ ΔΙΚΤΥΑ - 2η ΕΡΓΑΣΙΑ

Ζαρχαδούλας Ευάγγελος - AEM: 10649

Δεκέμβριος 2024

1 Εισαγωγή

Σε αυτή την εργασία υλοποιείται ένα Support Vector Machine το οποίο εκπαιδεύεται για να επιλύει τον διαχωρισμό όλων των κλάσεων που υπάρχουν στο Dataset Cifar-10.

1.1 Support Vector Machine

Τα Support Vector Machines αποτελούν έναν αλγόριθμο μηχανικής μάθησης που χρησιμοποιείται για προβλήματα ταξινόμησης. Στόχος αυτού του αλγορίθμου είναι να βρει ένα υπερεπίπεδο που διαχωρίζει τα δεδομένα σε διαφορετικές κατηγορίες μεγιστοποιώντας το περιθώριο μεταξύ των κατηγοριών. Τα SVMs εστιάζουν στα δεδομένα που βρίσκονται κοντά στο υπερεπίπεδο, τα οποία ονομάζονται support vectors, και χρησιμοποιούνται για τον καθορισμό της θέσης και του προσανατολισμού του υπερεπιπέδου. Όταν τα δεδομένα δεν είναι γραμμικά διαχωρίσιμα, εφαρμόζεται η μέθοδος του πυρήνα (kernel trick), η οποία επιτρέπει τη χαρτογράφησή τους σε έναν υψηλότερης διάστασης χώρο, ώστε να διαχωριστούν γραμμικά. Οι πυρήνες (kernels) που χρησιμοποιούνται στην εργασία είναι ο γραμμικός πυρήνας (linear kernel), ο πολυωνυμικός πυρήνας (polynomial kernel), καθώς και ο RBF (Radial Basis Function) πυρήνας, οι οποίοι αποτελούν και τους συνηθέστερους πυρήνες.

1.2 Αναλυτικότερη ανάλυση των SVMs

Ο αλγόριθμος των SVMs βασίζεται σε μαθηματικές αρχές για την επίτευξη του βέλτιστου διαχωριστικού επιπέδου που διαχωρίζει τα δεδομένα σε διαφορετικές κατηγορίες.

Για ένα σύνολο δεδομένων (x_i, y_i) , όπου $x_i \in R^n$ είναι το διάνυσμα χαρακτηριστικών του δείγματος i και $y_i \in \{-1, 1\}$ είναι η ετικέτα κατηγορίας, ο στόχος του SVM είναι να βρει ένα γραμμικό υπερεπίπεδο που διαχωρίζει τα δεδομένα. Το υπερεπίπεδο ορίζεται από την εξίσωση:

$$w \cdot x + b = 0$$

όπου: $w \in R^n$ είναι το διάνυσμα βαρών που καθορίζει την κατεύθυνση του υπερεπιπέδου και $b \in R$ είναι το bias.

Τα δεδομένα διαχωρίζονται στις δύο πλευρές του υπερεπιπέδου ως εξής:

- Για $y_i = +1$: $w \cdot x_i + b > 0$,
- Για $y_i = -1$: $w \cdot x_i + b < 0$.

Το περιθώριο M ορίζεται ως η ελάχιστη απόσταση από το υπερεπίπεδο στα πλησιέστερα σημεία δεδομένων. Το SVM επιδιώκει να μεγιστοποιήσει το περιθώριο, καθώς αυτό βελτιώνει τη γενίκευση του μοντέλου.

Για να μεγιστοποιηθεί το περιθώριο, ελαχιστοποιείται το $\|w\|^2$:

$$\min_{w,b} \frac{1}{2} \|w\|^2$$

υπό τον περιορισμό:

$$y_k(w^T x_k + b) \geq 1, \forall k$$

Όπου:

- $w \in R^n$: το διάνυσμα των βαρών,
- $b \in R$: το bias,
- $x_k \in R^n$: το διάνυσμα χαρακτηριστικών του k -οστού δείγματος,
- $y_k \in \{-1, 1\}$: η ετικέτα κατηγορίας.

Για να λύσουμε το παραπάνω πρόβλημα, χρησιμοποιούμε τους πολλαπλούς πολλαπλασιαστές Lagrange. Η Lagrangian συνάρτηση διαμορφώνεται ως:

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{k=1}^N \alpha_k [y_k(w^T x_k + b) - 1]$$

Όπου:

- $\alpha_k \geq 0$: οι πολλαπλασιαστές Lagrange για κάθε περιορισμό.

Η βέλτιστη λύση βρίσκεται με μηδενισμό των παραγώγων της Lagrangian ως προς τις παραμέτρους w και b :

1. Παράγωγος ως προς w :

$$\frac{\partial \mathcal{L}}{\partial w} = w - \sum_{k=1}^N \alpha_k y_k x_k = 0 \implies w = \sum_{k=1}^N \alpha_k y_k x_k$$

2. Παράγωγος ως προς b :

$$\frac{\partial \mathcal{L}}{\partial b} = - \sum_{k=1}^N \alpha_k y_k = 0$$

Αντικαθιστώντας το w στη Lagrangian, η βελτιστοποίηση διαμορφώνεται ως εξής:

$$\max_{\alpha} \left[\sum_{k=1}^N \alpha_k - \frac{1}{2} \sum_{k=1}^N \sum_{j=1}^N \alpha_k \alpha_j y_k y_j (x_k^T x_j) \right]$$

υπό τους περιορισμούς:

$$\alpha_k \geq 0, \quad \sum_{k=1}^N \alpha_k y_k = 0$$

Η λύση α_k υπολογίζεται λύνοντας πρόβλημα τετραγωνικού προγραμματισμού.

- Τα βάρη w υπολογίζονται ως:

$$w = \sum_{k=1}^N \alpha_k y_k x_k$$

- Το bias b υπολογίζεται από τα support vectors:

$$y_k(w^T x_k + b) = 1 \implies b = y_k - w^T x_k$$

Ωστόσο, τα δεδομένα συνήθως δεν είναι απολύτως διαχωρίσιμα, όποτε εισάγονται μεταβλητές χαλάρωσης ϵ_i ,

όπου $\epsilon_i \geq 0$ μετράει το πόσο ένα δείγμα x_i παραβιάζει το περιθώριο ή ταξινομείται λανθασμένα.

Επομένως το πρόβλημα μετατρέπεται ως εξής:

$$\min_{w, b, \epsilon_i} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \epsilon_i$$

υπό τους περιορισμούς:

$$y_i(w^T x_i + b) \geq 1 - \epsilon_i, \quad \epsilon_i \geq 0, \quad \forall i$$

όπου C είναι μια θετική παράμετρος η οποία ελέγχει το trade-off μεταξύ του μεγέθους του περιθωρίου και του κόστους των σφαλμάτων ταξινόμησης. Έχοντας μια μεγάλη τιμή για την παράμετρο C , το μοντέλο προσπαθεί να αποφύγει σφάλματα ταξινόμησης στο σετ εκπαίδευσης επιλέγοντας έτσι μικρότερο περιθώριο για να ταξινομήσει σωστά τα δεδομένα εκπαίδευσης, με αποτέλεσμα το μοντέλο να μπορεί να παρουσιάσει υπερπροσαρμογή (overfitting) και να μην γενικεύει καλά σε νέα δεδομένα.

Για μικρή τιμή του C , το μοντέλο επιτρέπει περισσότερα σφάλματα ταξινόμησης στο σετ εκπαίδευσης, επιλέγοντας έτσι ένα μεγαλύτερο περιθώριο αποδεχόμενο ότι κάποια σημεία μπορεί να βρίσκονται εντός του περιθωρίου ή να ταξινομούνται λανθασμένα. Το μοντέλο παρουσιάζει καλύτερη γενίκευση (καλύτερη απόδοση σε νέα δεδομένα), αλλά μπορεί να υποεκπαιδευτεί (underfitting).

Όταν τα δεδομένα δεν είναι γραμμικά διαχωρίσιμα, το SVM χρησιμοποιεί το **Kernel Trick**. Η εφαρμογή των kernels (συναρτήσεις πυρήνα) επιτρέπει στο SVM να λειτουργήσει σε έναν υψηλότερης διάστασης χώρο χωρίς να χρειάζεται να υπολογίσει ρητά τις συντεταγμένες αυτού του χώρου. Το Kernel Trick είναι μια μαθηματική ιδιότητα που επιτρέπει να αντικατασταθεί το εσωτερικό γινόμενο $(\phi(x_i)^T \phi(x_j))$ στον υψηλότερο χώρο από μια συνάρτηση πυρήνα $K(x_i, x_j)$, η οποία υπολογίζεται απευθείας στον αρχικό χώρο των χαρακτηριστικών.

Έτσι, αντί να εφαρμόσουμε μια συνάρτηση $\phi(x)$ για να μεταφέρουμε τα δεδομένα σε έναν υψηλότερης διάστασης χώρο, χρησιμοποιούμε μία συνάρτηση πυρήνα K , όπου:

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

Αυτό εξαλείφει την ανάγκη να υπολογίσουμε ρητά τη χαρτογράφηση $\phi(x)$, εξοικονομώντας χρόνο και υπολογιστική ισχύ.

Οι πιο συνηθισμένοι πυρήνες οι οποίοι χρησιμοποιούνται στα πλαίσια αυτής της εργασίας είναι οι εξής:

1. Γραμμικός Πυρήνας:

$$K(x_i, x_j) = x_i^T x_j$$

2. Πολυωνυμικός Πυρήνας:

$$K(x_i, x_j) = (\gamma x_i^T x_j + c)^d$$

3. RBF (Radial Basis Function) Πυρήνας:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

Τέλος, η πρόβλεψη για ένα νέο δείγμα x γίνεται με:

$$f(x) = \sum_{i=1}^N \alpha_i y_i K(x_i, x) + b$$

και η κατηγορία του δείγματος είναι:

$$\hat{y} = \text{sign}(f(x))$$

2 Αλγόριθμος

Χρησιμοποιώ τις βιβλιοθήκες:

- **NumPy** Για μαθηματικούς υπολογισμούς με πίνακες και αριθμητικά δεδομένα
- **OS** Για την αλληλεπίδραση με το σύστημα - διαχείριση αρχείων
- **Pickle** Για την ανάγνωση των δεδομένων CIFAR-10 τα οποία είναι αποθηκευμένα σε δυαδικά αρχεία.

- **Pickle** Για την ανάγνωση των δεδομένων CIFAR-10 τα οποία είναι αποθηκευμένα σε δυαδικά αρχεία.
- **sklearn** Βιβλιοθήκη για Μηχανική Μάθηση η οποία περιέχει:
 - **SVC** Η κλάση SVC χρησιμοποιείται για την υλοποίηση ενός Support Vector Classifier (SVC) και προσφέρει εκπαίδευση και πρόβλεψη με SVM για ταξινόμηση και υποστήριξη διαφορετικών kernels
 - **PCA (Principal Component Analysis)** Η PCA χρησιμοποιείται για μείωση διάστασης των δεδομένων και προσφέρει την διατήρηση των σημαντικότερων χαρακτηριστικών των δεδομένων και την ελαχιστοποίηση της πληροφορίας που χάνεται
 - **Accuracy Score** Η συνάρτηση accuracy_score μετρά την ακρίβεια ενός μοντέλου.
 - **Shuffle** Η συνάρτηση shuffle ανακατεύει τα δεδομένα και διασφαλίζει ότι τα δεδομένα εκπαίδευσης είναι τυχαία και δεν έχουν κάποια τάξη.
- **Time** Η βιβλιοθήκη time χρησιμοποιείται για τη μέτρηση του χρόνου εκτέλεσης του προγράμματος.

2.1 Φόρτωση Δεδομένων Cifar-10

CIFAR-10:

- **Είσοδος:** Το path του φακέλου με τα αρχεία των δεδομένων.
- **Έξοδος:**
 - **X_train:** Δεδομένα εκπαίδευσης (διανύσματα χαρακτηριστικών).
 - **y_train:** Ετικέτες κατηγοριών για τα δεδομένα εκπαίδευσης.
 - **X_test:** Δεδομένα δοκιμής.
 - **y_test:** Ετικέτες κατηγοριών για τα δεδομένα δοκιμής.
 - **label_names:** Ονόματα των κατηγοριών.

Η συνάρτηση load_cifar10_data ανοίγει και διαβάζει τα αρχεία δεδομένων με το pickle, φορτώνει τα δεδομένα εκπαίδευσης και δοκιμής και κανονικοποιεί τα δεδομένα διαιρώντας με το 255 ώστε οι τιμές να βρίσκονται στο [0, 1].

2.2 Εφαρμογή PCA για μείωση διάστασης

```
# 2. Μείωση διάστασης με PCA
print("Εφαρμογή PCA για μείωση διάστασης...")
pca = PCA(0.92)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)
```

Σχήμα 1: PCA

Αν έχουμε δεδομένα υψηλής διάστασης, πολλά από τα χαρακτηριστικά τους μπορεί να είναι περιττά. Το PCA αναλύει τα δεδομένα για να εντοπίσει τις πιο σημαντικές διαστάσεις και αφαιρεί τις μη σημαντικές διαστάσεις, μειώνοντας έτσι την πολυπλοκότητα.

Πλεονεκτήματα:

- Μειώνει την πολυπλοκότητα των δεδομένων.
- Εντοπίζει και απομακρύνει τον θόρυβο.
- Επιταχύνει αλγορίθμους μηχανικής μάθησης.
- Επιτρέπει την οπτικοποίηση δεδομένων υψηλής διάστασης.

Μειονεκτήματα:

- Χάνει πληροφορία αν μειωθούν πάρα πολλές διαστάσεις.
- Οι νέες συνιστώσες δεν είναι εύκολα ερμηνεύσιμες (δεν αντιστοιχούν σε αρχικά χαρακτηριστικά).
- Υποθέτει ότι τα δεδομένα κατανέμονται γραμμικά (μπορεί να μην αποδίδει καλά σε μη γραμμικά δεδομένα).

Στην εικόνα βλέπουμε ότι θα διατηρηθεί το 92% της διασποράς των δεδομένων.

Η `fit_transform` εκτελεί PCA στα δεδομένα εκπαίδευσης.

Η `transform` εφαρμόζει PCA στα δεδομένα δοκιμής.

2.3 Ορισμός και Εκπαίδευση του SVM

Η κλάση SVC από το `sklearn.svm` δημιουργεί έναν ταξινομητή SVM και παίρνει ως ορίσματα τα εξής:

- `kernel`: Καθορίζει τη συνάρτηση πυρήνα (π.χ. γραμμικός, RBF, πολυωνυμικός).
- `C`: Ελέγχει το trade-off μεταξύ του μεγάλου περιθωρίου και των σφαλμάτων ταξινόμησης.
- `gamma`: Ρυθμίζει την κλίμακα για πυρήνες όπως RBF ή πολυωνυμικό.
- `degree`: Ο βαθμός του πολυωνυμικού πυρήνα (αν χρησιμοποιείται).
- `decision_function_shape`:
 - `'ovo'` (One-vs-One): Δημιουργεί ταξινομητές για κάθε ζεύγος κατηγοριών.
 - `'ovr'` (One-vs-Rest): Δημιουργεί έναν ταξινομητή για κάθε κατηγορία έναντι όλων των άλλων.

Gamma: Η παράμετρος `gamma` είναι μια σημαντική υπερπαράμετρος στους μη γραμμικούς πυρήνες του Support Vector Machine (SVM), όπως ο Radial Basis Function (RBF) και ο πολυωνυμικός πυρήνας. Επηρεάζει άμεσα τη συμπεριφορά του μοντέλου και την ικανότητά του να προσαρμόζεται στα δεδομένα.

Για μεγάλη τιμή του `gamma` το μοντέλο θεωρεί τα σημεία που βρίσκονται πιο κοντά μεταξύ τους πιο σημαντικά και η επιρροή ενός σημείου είναι πολύ τοπική, επηρεάζοντας μόνο τα σημεία που βρίσκονται κοντά του. Αποτέλεσμα αυτού είναι το μοντέλο γίνεται πολύπλοκο και μπορεί να προσαρμοστεί στον θόρυβο.

Για μικρή τιμή του `gamma` τα σημεία έχουν μεγαλύτερη εμβέλεια επιρροής, επηρεάζοντας και πιο μακρινά σημεία και το μοντέλο δίνει έμφαση στη συνολική δομή των δεδομένων. Για πολύ μικρές τιμές του `gamma` το μοντέλο έχει κίνδυνο να χάσει λεπτομέρειες.

Η συνάρτηση `fit` της κλάσης SVC είναι υπεύθυνη για την εκπαίδευση του μοντέλου στα δεδομένα εκπαίδευσης, χρησιμοποιώντας τη θεωρία των SVM. Δηλαδή λύνει ένα πρόβλημα βελτιστοποίησης προσαρμόζοντας τις παραμέτρους α_i , w , και b και καθορίζει τα support vectors.

2.4 Αξιολόγηση του Μοντέλου

Η αξιολόγηση πραγματοποιείται τόσο στα δεδομένα εκπαίδευσης όσο και στα δεδομένα δοκιμής

```
# Αξιολόγηση του μοντέλου
y_pred_train = svm_rbf.predict(X_train_pca)
y_pred_test = svm_rbf.predict(X_test_pca)
```

Σχήμα 2: Αξιολόγηση του μοντέλου στην περίπτωση χρήσης του Rbf kernel

Η μέθοδος `predict` χρησιμοποιεί το υπερ-επίπεδο που προέκυψε από την εκπαίδευση του SVM για να αποφασίσει σε ποια πλευρά του υπερ-επιπέδου βρίσκεται κάθε σημείο δεδομένων και επιστρέφει την ετικέτα της κατηγορίας που αντιστοιχεί σε αυτήν την πλευρά.

Στην περίπτωση του **One-vs-one** και έχοντας 10 κλάσεις επειδή χρησιμοποιούμε το dataset Cifar-10 δημιουργούνται 45 δυαδικοί ταξινομητές. Για κάθε εισαγόμενο δείγμα κάθε δυαδικός ταξινομητής προβλέπει σε ποια από τις δύο κατηγορίες ανήκει το δείγμα και η κατηγορία με τις περισσότερες ψήφους επιστρέφεται ως πρόβλεψη.

Στην περίπτωση του **One-vs-rest** έχοντας 10 κλάσεις δημιουργούνται 10 δυαδικοί ταξινομητές, ένας για κάθε κατηγορία έναντι όλων των άλλων. Για κάθε εισαγόμενο δείγμα υπολογίζεται η απόσταση του δείγματος από το υπερ-επίπεδο κάθε ταξινομητή και η κατηγορία με τη μεγαλύτερη απόσταση από το υπερ-επίπεδο επιστρέφεται ως πρόβλεψη.

Η predict βασίζεται στην εξής εξίσωση απόφασης:

$$f(x) = \sum_{i=1}^N \alpha_i y_i K(x_i, x) + b$$

Αν $f(x) > 0$, τότε η πρόβλεψη είναι $y = +1$.

Αν $f(x) < 0$, τότε η πρόβλεψη είναι $y = -1$.

Έπειτα με την χρήση της συνάρτησης `accuracy_score` αποθηκεύονται τα training και testing accuracies αντίστοιχα.

2.5 Εκτυπώσεις αποτελεσμάτων

Στο τέλος έχω την εκτύπωση των accuracies και τον χρόνο εκπαίδευσης καθώς επίσης, στο τέλος του προγράμματος, στις περιπτώσεις που δοκιμάζω διάφορες τιμές για τις παραμέτρους της κλάσης SVC εκτυπώνω για ποιές παραμέτρους έχω λάβει το καλύτερο training accuracy.

3 Πειραματισμοί

Σε αυτό το κομμάτι την αναφοράς παρουσιάζονται τα αποτελέσματα των δοκιμών που λήφθηκαν από τους πειραματισμούς τόσο στην επιλογή του πυρήνα που χρησιμοποιούμε, όσο και στις διαφορετικές τιμές των παραμέτρων εκπαίδευσης.

3.1 Εφαρμογή Γραμμικού Πυρήνα (Linear Kernel)

Οι πειραματισμοί ξεκινάνε με την χρήση του γραμμικού πυρήνα χρησιμοποιώντας αρχικά PCA που διατηρεί το 92% της διασποράς των δεδομένων και ταξινομητή one-vs-one.

Ο τρόπος που προσεγγίζεται το πρόβλημα είναι ο εξής:

Ξεκινάω τους πειραματισμούς με ένα μέρος των δεδομένων του data set έτσι ώστε να περιορίσω τις τιμές των παραμέτρων που είναι υποψήφιες να μου δώσουν το καλύτερο testing accuracy σε πιο σύντομο χρονικό διάστημα, καθώς η επίτευξη του μέγιστου testing accuracy είναι το ζητούμενο των πειραματισμών.

Η παράμετρος με την οποία πειραματίζομαι όταν χρησιμοποιείται ο γραμμικός πυρήνας είναι η C της οποίας ο ορίσμος έχει δωθεί παραπάνω.

3.1.1 5000 training data και 1000 testing data

Για αυτό τον όγκο των δεδομένων training και testing δοκιμάζω τις τιμές

$C = [0.001, 0.01, 0.1, 1, 10, 50, 100]$

και έλαβα τα εξής αποτελέσματα:

C	Training Accuracy	Testing Accuracy	Χρόνος εκπαίδευσης
0.001	39.10%	36.20%	1.09 sec
0.01	46.44%	37.20%	1.01 sec
0.1	49.94%	36.10%	1.82 sec
1	51.18%	34.70%	15.34 sec
10	51.20%	34.20%	184.11 sec
50	51.32%	34.40%	972.17 sec
100	51.28%	34.60%	1904.74 sec

Πίνακας 1: Linear Kernel 1η δοκιμή

Από τον παραπάνω πίνακα παρατηρώ ότι λαμβάνω καλύτερα testing accuracy για μικρότερα C. Επίσης παρατηρείται ότι αυξάνοντας το C, αυξάνεται και ο χρόνος εκπαίδευσης, διότι με υψηλό C το μοντέλο δίνει έμφαση στο να ταξινομεί σωστά τα δεδομένα εκπαίδευσης, μειώνοντας τα σφάλματα ταξινόμησης και αυξάνοντας έτσι την υπολογιστική πολυπλοκότητα. Τέλος, παρατηρούμε ότι όσο αυξάνεται το C, αυξάνεται και το training accuracy καθώς το μοντέλο αρχίζει και προσαρμόζεται περισσότερο στα δεδομένα εκπαίδευσης, χάνοντας έτσι σιγά σιγά την γενικότητα του, εξού και η μείωση του testing accuracy.

Επομένως τώρα κρατώ λιγότερες τιμές του C με τις οποίες θα πειραματιστώ και αυξάνω τον αριθμό των δεδομένων.

3.1.2 15000 training data και 3000 testing data

Για αυτό τον όγκο των δεδομένων training και testing δοκιμάζω τις τιμές

C = [0.0001, 0.001, 0.01, 0.1, 1, 10]

C	Training Accuracy	Testing Accuracy	Χρόνος εκπαίδευσης
0.0001	34.08%	34.33%	0.19 min
0.001	40.29%	39.57%	0.15 min
0.01	43.91%	40.30%	0.14 min
0.1	44.37%	40.17%	0.24 min
1	44.66%	39.90%	1.20 min
10	44.59%	40.03%	10.38 sec

Πίνακας 2: Linear Kernel 2η δοκιμή

Συνεχίζω την σταδιακή μείωση των C, αφαιρώντας τις τιμές που μου δίνουν τις συγκριτικά χειρότερες αποδόσεις, αυξάνοντας παράλληλα τον αριθμό των δεδομένων που χρησιμοποιώ.

3.1.3 25000 training data και 5000 testing data

Για αυτό τον όγκο των δεδομένων training και testing δοκιμάζω τις τιμές

C = [0.01, 0.1, 1, 10]

C	Training Accuracy	Testing Accuracy	Χρόνος εκπαίδευσης
0.01	42.96%	41.08%	0.45 min
0.1	43.29%	40.86%	0.72 min
1	43.42%	40.76%	2.61 min
10	43.42%	40.86%	19.34 sec

Πίνακας 3: Linear Kernel 3η δοκιμή

Παρατηρώ ότι οι τιμές που έλαβα για το testing accuracy σε αυτή τη δοκιμή είναι όλες παραπλήσιες, οπότε αυτές οι τιμές των C είναι οι υποψήφιες για να μου δώσουν το μέγιστο testing accuracy, οπότε κάνω την τελευταία δοκιμή με όλο το πλήθος των δεδομένων

3.1.4 50000 training data και 10000 testing data

Για αυτό τον όγκο των δεδομένων training και testing δοκιμάζω τις τιμές

$$C = [0.01, 0.1, 1, 10]$$

C	Training Accuracy	Testing Accuracy	Χρόνος εκπαίδευσης
0.01	42.59%	41.49%	3.55 min
0.1	42.84%	41.26%	5.13 min
1	42.87%	41.31%	12.6 min
10	42.88%	41.31%	65.1 sec

Πίνακας 4: Linear Kernel 4η δοκιμή

Τελικά παρατηρώ ότι για Linear Kernel το SVM λειτουργεί αποδοτικότερα για $C=0.01$ και δίνει απόδοση 41.49% για one-vs-one classification και $PCA=0.92\%$

Παρατηρούμε επίσης ότι όσο αυξάνω τα δεδομένα στις δοκιμές μειώνεται το training accuracy, πράγμα που μας οδηγεί στο λογικό συμπέρασμα ότι αυξάνοντας τα δεδομένα, το μοντέλο υπερεκπαιδεύεται λιγότερο.

3.1.5 Πειραματισμοί στο PCA

Από τα παραπάνω πειράματα παρατηρούμε ότι όταν χρησιμοποιούμε PCA διατηρώντας το 0.92% της διασποράς των δεδομένων λαμβάνουμε testing accuracy = 41.49% για $C=0.01$. Είναι λογικό ότι αν αυξήσουμε το ποσοστό του PCA θα έχουμε αύξηση και στο testing accuracy καθώς θα έχουμε περισσότερες συνιστώσες και έτσι το μοντέλο θα έχει περισσότερες πληροφορίες να χρησιμοποιήσει.

Πράγματι, όταν αφαιρώ εξολοκλήρου το PCA, δηλαδή όταν διατηρώ το 100% της διασποράς των δεδομένων, για την ίδια τιμή C , έχω το εξής:

C	Training Accuracy	Testing Accuracy	Χρόνος εκπαίδευσης
0.01	47.08%	42.41%	40.49 min

Πίνακας 5: No PCA

Παρατηρούμε ότι η αφαίρεση του PCA μας δίνει ελαφρώς υψηλότερο testing accuracy και υψηλότερο training accuracy. Το υψηλότερο training accuracy μας δίνει ότι τα επιπλέον χαρακτηριστικά που είχαν αφαιρεθεί με την χρήση του PCA, βοηθούν το μοντέλο να μάθει καλύτερα τα δεδομένα δοκιμής και να αρχίσει να υπερεκπαιδεύεται σε αυτά

Έπειτα, μειώνω το ποσοστό του PCA σε 90% και λαμβάνω το εξής:

C	Training Accuracy	Testing Accuracy	Χρόνος εκπαίδευσης
0.01	41.95%	41.08%	2.53 min

Πίνακας 6: PCA 0.9

Βλέπουμε ότι το testing accuracy μειώνεται, άρα συμπεραίνουμε ότι έχει χαθεί πολύτιμη πληροφορία από το PCA, ενώ μειώνεται και το training accuracy, δηλαδή η πληροφορία που χάθηκε βοήθησε στην γενίκευση.

3.2 Εφαρμογή Πολυωνυμικού Πυρήνα (Polynomial Kernel)

Η τακτική που ακολουθήθηκε και για αυτόν τον πυρήνα είναι η ίδια με πριν, με μόνη διαφορά να είναι ότι στην προκειμένη περίπτωση θα πειραματιστώ με δυο ακόμη παραμέτρους, τον βαθμό του πολυωνύμου και το gamma.

Λόγω των πολλών συνδυασμών θα γίνει μια σύντομη περιγραφή των αποτελεσμάτων που έλαβα για λιγότερα δεδομένα προσπαθώντας να καταλήξω στις τιμές του βαθμού του πολυωνύμου, του C και του gamma που θα μου δώσουν το καλύτερο testing accuracy και θα δωθεί πινακάκι στο τέλος με τα τελικά αποτελέσματα.

3.2.1 5000 training data και 1000 testing data

Για αυτό το αρχικό πλήθος δεδομένων δοκιμάστηκαν οι εξής τιμές:

Βαθμός Πολυωνύμου = [2,3,4]

gamma = [0.001, 0.01, 0.1, 1, 10, 50, 100]

C = [0.001, 0.01, 0.1, 1, 10, 50, 100]

Παρατήρησα ότι έχω τις καλύτερες αποδόσεις για βαθμό πολυωνύμου 2 και 3, για gamma μεταξύ 0.01-1 και C 0.001-10.

3.2.2 25000 training data και 5000 testing data

Παίρνοντας τις παραμέτρους που μου έδωσαν τα καλύτερα αποτελέσματα στο προηγούμενο πείραμα παίρνω τα εξής:

Παρατηρώ ότι έχω τις καλύτερες αποδόσεις για βαθμό πολυωνύμου 2 ενώ δεν κατάφερα να αποκλείσω κάποιες τιμές των C και gamma καθώς τα αποτελέσματα που έλαβα είναι αρκετά παρόμοια μεταξύ τους. Οπότε συνεχίζω στην τελευταία δοκιμή για όλα τα δεδομένα μου για

C = [0.001, 0.01, 0.1, 1, 10]

gamma = [0.01, 0.1, 1]

και βαθμό πολυωνύμου 2.

3.2.3 50000 training data και 10000 testing data

Για τις μεταβλητές που αναφέρθηκαν παραπάνω έλαβα:

Για C=0.001

gamma	Training Accuracy	Testing Accuracy	Χρόνος εκπαίδευσης
0.01	19.32%	20.11%	6.5 min
0.1	47.60%	43.56%	4.48 min
1	84.07%	49.13%	4.48 min

Πίνακας 7: C=0.001

Για C=0.01

gamma	Training Accuracy	Testing Accuracy	Χρόνος εκπαίδευσης
0.01	36.46%	32.44%	5.31 min
0.1	64.42%	49.53%	3.49 min
1	97.75%	46.94%	4.48 min

Πίνακας 8: C=0.01

Για C=0.1

gamma	Training Accuracy	Testing Accuracy	Χρόνος εκπαίδευσης
0.01	47.60%	43.57%	4.41 min
0.1	84.07%	49.15%	4.48 min
1	99.96%	45.44%	9.25 min

Πίνακας 9: C=0.1

Για C=1

gamma	Training Accuracy	Testing Accuracy	Χρόνος εκπαίδευσης
0.01	64.42%	49.53%	4 min
0.1	97.75%	46.93%	8.4 min
1	100%	45.22%	9.35 min

Πίνακας 10: C=1

Για $C=10$

gamma	Training Accuracy	Testing Accuracy	Χρόνος εκπαίδευσης
0.01	84.07%	49.14%	4.57 min
0.1	99.96%	45.43%	9.32 min
1	100%	45.27%	9.33 min

Πίνακας 11: $C=10$

Επομένως, όπως παρατηρούμε, έχω την καλύτερη απόδοση, δηλαδή το καλύτερο testing accuracy για βαθμό πολωνύμου 2, για $C=0.01$ και $\text{gamma}=0.1$

Παρατηρούμε επίσης ότι όσο αυξάνεται το gamma και το C , το training accuracy παίρνει πολύ μεγάλες τιμές που πολλές φορές αγγίζουν και το 100%, το οποίο είναι ξεκάθαρο δείγμα overfitting. Αυτό συμβαίνει διότι όσο αυξάνεται το gamma το μοντέλο γίνεται πιο ευαίσθητο σε μικρές διαφορές μεταξύ των δειγμάτων και αυτό επιτρέπει στο SVM να προσαρμόζεται σχεδόν τέλεια στα δεδομένα εκπαίδευσης ενώ αυξάνοντας το C το SVM επιβάλλει αυστηρότερους περιορισμούς, μειώνοντας τον αριθμό των σφαλμάτων ταξινόμησης στο training set. Ωστόσο, όσο το testing accuracy παραμένει υψηλό, δεν είναι κάτι που θα πρέπει να μας ανησυχεί.

3.2.4 Πειραματισμοί στο PCA

Όπως και στην περίπτωση του γραμμικού πυρήνα, έτσι και εδώ, δοκίμασα τόσο το να βγάλω το PCA τόσο και να μειώσω το ποσοστό του για τις τιμές των παραμέτρων που μου έδωσαν τις καλύτερες επιδόσεις και έλαβα τα εξής:

degree	C	gamma	Training Accuracy	Testing Accuracy	Χρόνος εκπαίδευσης
2	0.01	0.1	83.25%	51.97%	38.16 min

Πίνακας 12: No PCA

degree	C	gamma	Training Accuracy	Testing Accuracy	Χρόνος εκπαίδευσης
2	0.01	0.1	61.36%	49.15%	2.45 min

Πίνακας 13: PCA 0.9

Βλέπουμε ότι έχουμε τα ίδια συμπεράσματα με τον γραμμικό πυρήνα όσον αφορά το PCA.

3.3 Εφαρμογή Πυρήνα RBF

Ίδια τεχνική ακολουθήθηκε και για τον RBF kernel με τις μεταβλητές με τις οποίες πειραματίζομαι να είναι το C και το gamma .

3.3.1 5000 training data και 1000 testing data

Οι αρχικές τιμές που θέτω στο C και gamma είναι οι:

$C = [0.001, 0.01, 0.1, 1, 10, 50, 100]$

$\text{gamma} = [0.001, 0.01, 0.1, 1, 10, 50, 100]$

Παρατηρώ ότι τα καλύτερα αποτελέσματα σημειώνονται για χαμηλές τιμές του gamma 0.001-0.1 και για υψηλότερες τιμές του C 1-100

3.3.2 15000 training data και 3000 testing data

Για αυτό το σύνολο των δεδομένων κάνω δοκιμές για τις τιμές:

$C = [1, 10, 50, 100]$

$\text{gamma} = [0.001, 0.01, 0.1]$

C	gamma	Training Accuracy	Testing Accuracy	Χρόνος εκπαίδευσης
1	0.001	46.74%	44.27%	8 sec
1	0.01	78.55%	50.20%	9 sec
1	0.1	99.98%	22.13%	16 sec
10	0.001	59.93%	48.13%	8 sec
10	0.01	99.50%	49.33%	14 sec
10	0.1	99.98%	24.03%	16 sec
50	0.001	76.36%	49.57%	10 sec
50	0.01	100%	48.53%	14 sec
50	0.1	100%	24.03%	16 sec
100	0.001	84.75%	48.87%	12 sec
100	0.01	100%	48.43%	14 sec
100	0.1	100%	24.03%	16 sec

Πίνακας 14: 2η δοκιμή RBF

Παρατηρώ ότι για $\gamma=0.1$ το testing accuracy είναι αρκετά χαμηλό επομένως το αποκλείω.

3.3.3 50000 training data και 10000 testing data

C	gamma	Training Accuracy	Testing Accuracy	Χρόνος εκπαίδευσης
1	0.001	49.06%	47.40%	3.33 min
1	0.01	77.22%	55.39%	3.31 min
10	0.001	59.74%	52.54%	3.11 min
10	0.01	98.70%	56.15%	6.2 min
50	0.001	72.63%	55.13%	4.1 min
50	0.01	99.96%	55.61%	6.27 min
100	0.001	79.58%	55.64%	4.36 min
100	0.01	100%	55.69%	6.19 min

Πίνακας 15: 3η δοκιμή RBF

Επομένως παρατηρώ ότι το καλύτερο testing accuracy παρατηρείται για τιμές παραμέτρων $C=10$ $\gamma=0.01$ και είναι 56.15%

3.3.4 Πειραματισμοί στο PCA

Όπως και με τους προηγούμενους 2 πυρήνες, έτσι και εδώ δοκιμάζω τον πυρήνα RBF χωρίς PCA και με μικρότερο ποσοστό PCA και παίρνω τα εξής:

C	gamma	Training Accuracy	Testing Accuracy	Χρόνος εκπαίδευσης
10	0.01	99.98%	56.70%	95.8 min

Πίνακας 16: No PCA

C	gamma	Training Accuracy	Testing Accuracy	Χρόνος εκπαίδευσης
10	0.01	98.09%	55.87%	4.1 min

Πίνακας 17: PCA 0.9

Και σε αυτή τη περίπτωση βλέπουμε ότι όσο μειώνεται το ποσοστό στο PCA, τόσο μειώνεται και το testing accuracy. Οι μεγαλύτερες αποδόσεις σημειώνονται χωρίς την χρήση PCA, άλλα ο χρόνος εκπαίδευσης καθιστούν τα μοντέλα αυτά απαγορευτικά στην πλειοψηφία των περιπτώσεων.

3.4 Συγκρίσεις Κατηριοποιητών

3.4.1 MLP με 1 κρυφό επίπεδο και χρήση Hinge Loss

Αυτό το MLP υλοποιείται όπως στην προηγούμενη εργασία με μόνη διαφορά την αντικατάσταση της Softmax και της Cross Entropy με την Hinge Loss.

Hinge loss: Το Hinge Loss είναι μια συνάρτηση απώλειας που υπολογίζει το περιθώριο μεταξύ της σωστής πρόβλεψης και των λανθασμένων.

Για δυαδική ταξινόμηση:

$$\text{Hinge Loss} = \max(0, 1 - y_{\text{true}} \cdot y_{\text{pred}})$$

```
def hinge_loss(y_true, y_pred):  
    return np.mean(np.maximum(0, 1 - y_true * y_pred))
```

Σχήμα 3: Hinge loss function

όπου:

- $y_{\text{true}} \in \{-1, +1\}$: Η πραγματική ετικέτα της κλάσης.
- $y_{\text{pred}} \in R$: Η πρόβλεψη του μοντέλου πριν από την ενεργοποίηση.

Ερμηνεία του Hinge Loss

1. Όταν $y_{\text{true}} \cdot y_{\text{pred}} \geq 1$:

- Το μοντέλο έχει κάνει σωστή πρόβλεψη με περιθώριο ≥ 1 .
- Η απώλεια είναι 0 (το μοντέλο έχει εκπληρώσει τον στόχο του).

2. Όταν $y_{\text{true}} \cdot y_{\text{pred}} < 1$:

- Το μοντέλο είτε έκανε λάθος πρόβλεψη είτε δεν είχε αρκετό περιθώριο.
- Η απώλεια είναι $1 - y_{\text{true}} \cdot y_{\text{pred}}$, που τιμωρεί το μοντέλο ανάλογα με το πόσο μακριά είναι η πρόβλεψη από το επιθυμητό αποτέλεσμα.

Στον κωδικα

Οι ετικέτες μετατρέπονται σε One-hot Encoding με -1 και +1 όπου η σωστή κλάση έχει τιμή +1 ενώ οι άλλες έχουν τιμή -1 και το hinge_loss υπολογίζει

$$\max(0, 1 - y_{\text{true},j} \cdot y_{\text{pred},j})$$

για κάθε κλάση j.

Ο μέσος όρος της απώλειας για όλες τις κλάσεις και όλα τα δείγματα υπολογίζεται ως εξής:

$$\text{Hinge Loss}_{\text{mean}} = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^{10} \max(0, 1 - y_{\text{true},ij} \cdot y_{\text{pred},ij})$$

```
def hinge_loss_derivative(y_true, y_pred):  
    grad = np.zeros_like(y_pred)  
    mask = (1 - y_true * y_pred > 0)  
    grad[mask] = -y_true[mask]  
    return grad
```

Σχήμα 4: Hinge loss derivative

Στο backward propagation το σφάλμα στην έξοδο είναι

$$\delta_{\text{output}} = \text{hinge_loss.derivative}(y_{\text{true}}, y_{\text{pred}})$$

όπου η παράγωγος του hinge loss λειτουργεί ως εξής:

- Αν $1 - y_{\text{true},j} \cdot y_{\text{pred},j} > 0$, η παράγωγος είναι $-y_{\text{true},j}$.
- Διαφορετικά, η παράγωγος είναι 0.

και τα gradients για τα βάρη και τα biases κάθε επιπέδου υπολογίζονται με βάση το σφάλμα (δ).

```
def preprocess_data(train_data, train_labels, test_data, test_labels):
    num_classes = 10
    train_labels_one_hot = -1 * np.ones((train_labels.size, num_classes))
    train_labels_one_hot[np.arange(train_labels.size), train_labels] = 1

    test_labels_one_hot = -1 * np.ones((test_labels.size, num_classes))
    test_labels_one_hot[np.arange(test_labels.size), test_labels] = 1

    return train_data, train_labels_one_hot, test_data, test_labels_one_hot
```

Σχήμα 5: One hot Encoding

3.4.2 Συγκρίσεις

Method	Training Accuracy	Testing Accuracy	Time
1-NN	-	35.39%	40.2 min
3-NN	-	33.03%	33.03 min
Nearest Centroid	-	27.74%	0.01 min
MLP	79.50%	52.18%	15.16 min
Linear Kernel	42.59%	41.49%	3.49 min
Polynomial Kernel	64.42%	49.53%	3.49 min
RBF Kernel	98.70%	56.15%	6.2 min

Πίνακας 18: 3η δοκιμή RBF

Συγκρίνοντας τους κατηγοριοποιητές Nearest Neighbor και Nearest Class Centroid με τα SVM, βλέπουμε την ξεκάθαρη υπεροχή των SVMs τόσο από άποψη accuracy τόσο και από άποψη χρόνου.

Όσον αφορά τον κατηγοριοποιητή MLP που χρησιμοποιεί Hinge loss με 1 κρυφό επίπεδο, βλέπουμε ότι έχει καλύτερη απόδοση από τα SVMs που χρησιμοποιούν πολυωνυμικό και γραμμικό πυρήνα, απαιτώντας βέβαια περισσότερο χρόνο. Ωστόσο, είναι ξεκάθαρη η υπεροχή του SVM που χρησιμοποιεί RBF kernel έναντι του MLP καθώς σε πολύ λιγότερο χρόνο καταφέρνει και επιτυγχάνει καλύτερο testing accuracy.

ΤΕΛΙΚΑ Βλέπουμε ότι το SVM που χρησιμοποιεί RBF kernel είναι αυτό που υπερισχύει από άποψη testing accuracy έναντι των κατηγοριοποιητών που αναφέρθηκαν προηγουμένως.

Ωστόσο, μπορεί να δει κανείς ότι η παρουσία του overfitting είναι αρκετά εντονότερη σε αυτόν τον κατηγοριοποιητή έναντι οποιουδήποτε άλλου, καθώς βλέπουμε ότι το training accuracy ξεπερνά το 98% για να επιτευχθεί το μέγιστο testing accuracy.